

COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course</i>	<i>Information security lab</i>
<i>Instructor</i>	<i>Mrs. Ambreen Gull</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>Fa24-bse-039</i>	<i>Muhammad Taha Niaz</i>

Question :1

Change 1 – Wrong modular exponentiation

Your power function:

```
return pow(a, b) % p
```

This first calculates **huge a^b** and then takes mod.
For real Diffie-Hellman this becomes extremely slow and insecure.

Python already has **built-in modular exponentiation**:

```
pow(a, b, p) # fast and secure
```

So we must fix the power function.

Change 2 – Wrong base case in power()

```
if b == 1:  
    return a
```

This is mathematically wrong for modular arithmetic.
Correct identity:

$$a^0 \bmod p = 1$$

But since we are using built-in pow(), we can remove the whole condition.

Change 3 – Program uses fixed private keys

Currently:

```
a = 4  
b = 3
```

Real Diffie-Hellman must use **random private keys**.
We will use:

```
import random  
a = random.randint(2, P-2)  
b = random.randint(2, P-2)
```

Updated Diffie-Hellman Code

```
import random  
  
# Fast modular exponentiation  
def power(a, b, p):  
    return pow(a, b, p)  
  
def main():  
    # Public values  
    P = 23  
    G = 9  
  
    print("Public prime P:", P)  
    print("Public primitive root G:", G)  
  
    # Private keys (random)  
    a = random.randint(2, P-2) # Alice private key  
    b = random.randint(2, P-2) # Bob private key  
  
    print("\nAlice private key:", a)  
    print("Bob private key:", b)  
  
    # Public keys  
    x = power(G, a, P) # Alice sends to Bob  
    y = power(G, b, P) # Bob sends to Alice  
  
    print("\nAlice public key:", x)
```

```
print("Bob public key:", y)

# Shared secret keys
ka = power(y, a, P)
kb = power(x, b, P)

print("\nSecret key for Alice:", ka)
print("Secret key for Bob:", kb)

if __name__ == "__main__":
    main()
```

Example Output (sample run)

Every run gives different keys due to randomness.

```
C:\Users\HP\IdeaProjects\PythonProject\PythonProject\
Public prime P: 23
Public primitive root G: 9

Alice private key: 20
Bob private key: 10

Alice public key: 2
Bob public key: 18

Secret key for Alice: 12
Secret key for Bob: 12

Process finished with exit code 0
```

Question no: 2

Latest Attack on Key Exchange Algorithms:

Harvest Now, Decrypt Later (HNDL) Attack

One of the **most recent and serious threats** to key-exchange systems is the **Harvest Now, Decrypt Later (HNDL)** attack.

Why this attack is important

Most secure communication today uses key exchange algorithms such as:

- Diffie–Hellman (DH)
- Elliptic Curve Diffie–Hellman (ECDH)
- RSA key exchange

These are secure against classical computers, but not secure against future quantum computers.

Idea of the Attack

The attacker does NOT break encryption today.

Instead they:

1. Intercept encrypted traffic (TLS, WhatsApp, VPN, etc.)
2. Store the encrypted data for years
3. Wait for powerful quantum computers
4. Use Shor's Algorithm to break the key exchange
5. Decrypt old conversations

This is why it is called:

Harvest Now → Decrypt Later

This attack specifically threatens Diffie-Hellman key exchange.

Why Diffie–Hellman Breaks with Quantum Computers

Diffie-Hellman security depends on a hard problem:

Discrete Logarithm Problem

Classical computers:

Hard to solve → secure

Quantum computers:

Shor's Algorithm solves it fast → broken

So attackers record traffic today and decrypt it in the future.

Demonstration of the Attack in Python

We will simulate how an attacker can break Diffie-Hellman using discrete logarithm brute force

(This represents the quantum attack conceptually.)

Step 1 — Normal Diffie-Hellman Key Exchange

```
import random
```

```
# Public values
```

```
p = 23    # prime number
```

```
g = 5     # generator
```

```
# Alice private key
```

```
a = random.randint(1, 10)
```

```
A = pow(g, a, p)
```

```
# Bob private key
```

```
b = random.randint(1, 10)
```

```
B = pow(g, b, p)
```

```
# Shared secret
```

```
alice_secret = pow(B, a, p)
```

```
bob_secret = pow(A, b, p)
```

```
print("Public Prime p:", p)
```

```
print("Public Generator g:", g)
```

```
print("Alice Public Key:", A)
```

```
print("Bob Public Key:", B)
```

```
print("Shared Secret:", alice_secret)
```

This simulates **secure communication**.

Step 2 — Attacker Breaking the Key Exchange

Attacker knows:

- p
- g
- A
- B

They try to find private key a from

$$A = g^a \bmod p$$

This is the **discrete logarithm attack**.

Attacker brute-force discrete logarithm

```
def break_diffie_hellman(p, g, public_key):  
    for x in range(p):  
        if pow(g, x, p) == public_key:  
            return x
```

```
# Attacker recovers Alice's private key  
recovered_a = break_diffie_hellman(p, g, A)
```

```
# Attacker computes shared secret  
attacker_secret = pow(B, recovered_a, p)
```

```
print("\nAttacker recovered Alice private key:", recovered_a)  
print("Attacker computed shared secret:", attacker_secret)
```